COMPUTER SCIENCE

Complete Concepts & Revision Guide

12 core subjects • Definitions • Models • Algorithms • Key comparisons

---

A practical introductory handbook for students studying computer science and information technology.

DBMS • Data Warehousing • Computer Networks • AI & ML • Web Technology • Network Security & Cryptography • DSA • Operating Systems • OOP • Software Engineering • Compiler Design • Computer Organization & Architecture

Use this as a structured overview and revision aid. Pair it with your course syllabus, lectures, coding practice, diagrams, and numerical exercises.

How to use this guide

Each chapter summarizes key vocabulary, models, methods, and trade-offs. Read for the big picture first, then revise by heading. For exams, practice diagrams, SQL queries, numerical problems, code tracing, and short-answer explanations alongside these notes.

Contents

1. Database Management Systems (DBMS)

2. Data Warehousing and Business Intelligence

3. Computer Networks

4. Artificial Intelligence and Machine Learning

5. Web Technology

6. Network Security and Cryptography

7. Data Structures and Algorithms (DSA)

8. Operating Systems (OS)

9. Object-Oriented Programming (OOP)

10. Software Engineering

11. Compiler Design

12. Computer Organization and Architecture (COA)

1. Database Management Systems (DBMS)

A DBMS is software that defines, stores, retrieves, updates, and protects data. It provides a structured way for applications and users to work with information.

Core vocabulary

- **Database:** organized related data. **Schema:** database structure. **Instance:** data at a particular moment. In the relational model, data is stored in tables with rows (tuples) and columns (attributes).
- **Keys:** a superkey uniquely identifies a row; a candidate key is a minimal superkey; a primary key is the chosen candidate key; a foreign key references a key in another table; a composite key uses multiple columns.
- **Integrity:** entity integrity means a primary key is not null; referential integrity keeps foreign-key references valid; domain constraints restrict allowed values.

SQL and relational operations

- **DDL:** CREATE, ALTER, DROP. **DML:** INSERT, UPDATE, DELETE. **DQL:** SELECT. **DCL:** GRANT, REVOKE. **TCL:** COMMIT, ROLLBACK, SAVEPOINT.
- SELECT retrieves data; WHERE filters rows; GROUP BY forms groups; HAVING filters groups; ORDER BY sorts results. Joins include INNER, LEFT, RIGHT, and FULL OUTER joins (support varies by system).
- Aggregate functions include COUNT, SUM, AVG, MIN, and MAX. A subquery is a query nested inside another query. A view is a saved query presented like a virtual table.

Normalization and transactions

- Normalization reduces undesirable redundancy and update anomalies. 1NF requires atomic values; 2NF removes partial dependency on a candidate key; 3NF removes problematic transitive dependencies; BCNF requires each determinant to be a candidate key.
- **ACID:** Atomicity (all or nothing), Consistency (valid rules remain true), Isolation (concurrent transactions are controlled), Durability (committed changes persist).
- Concurrency problems include dirty reads, non-repeatable reads, phantom reads, and lost updates. Locks, timestamps, MVCC, and serializable execution help manage concurrency. Deadlocks can be prevented, detected and recovered from, or handled with timeouts.

Indexes, recovery and design

- Indexes speed up selected queries but require storage and may slow writes. B+ trees support ordered and range searches; hash indexes are useful for equality lookups.
- ER diagrams model entities, attributes, relationships, cardinality (1:1, 1:N, M:N), and participation. M:N relationships are usually implemented with a junction table.
- Recovery uses logs, checkpoints, backups, and redo/undo operations. Good design balances data integrity, low redundancy, performance, security, and maintainability.

2. Data Warehousing and Business Intelligence

A data warehouse integrates data from multiple sources for historical analysis, reporting, and decision support.

OLTP versus OLAP

- **OLTP** systems support daily transactions, frequent small reads/writes, and current detailed data. **OLAP** systems support analytical queries, aggregations, trends, and large historical datasets.
- A warehouse pipeline commonly looks like: source systems → staging → cleaning/integration → warehouse → data marts/semantic layer → dashboards and analysis.

ETL, ELT and data quality

- **ETL** means Extract, Transform, Load. **ELT** loads data first and transforms it in the destination. Incremental loading processes only new/changed data; change data capture (CDC) identifies source changes.
- Data quality dimensions include accuracy, completeness, consistency, uniqueness, validity, and timeliness. Metadata describes field meanings, lineage, owners, refresh timing, and business definitions.
- A **data mart** serves a department or subject area. Governance includes access rules, retention, privacy, ownership, lineage, and consistent definitions of metrics.

Dimensional modeling

- A **fact table** stores measurable events (sales amount, units) and keys to dimensions. A **dimension** describes context such as customer, product, location, or date.
- **Star schema:** central fact table with denormalized dimensions; straightforward for analytics. **Snowflake schema:** dimensions are normalized into related tables.
- **Grain** defines exactly what one fact row represents. Slowly Changing Dimensions: Type 1 overwrites; Type 2 adds a row/version to preserve history; Type 3 stores limited prior values in extra columns.

3. Computer Networks

A computer network connects devices and allows them to exchange data using protocols, addressing, and communication links.

OSI reference model

- Layer 7 Application: services such as HTTP, DNS, SMTP. Layer 6 Presentation: data representation and encoding. Layer 5 Session: dialog coordination. Layer 4 Transport: TCP/UDP. Layer 3 Network: IP and routing. Layer 2 Data Link: frames, MAC, switching. Layer 1 Physical: signals, cables, radio, and bits.
- The TCP/IP model is often summarized as Link, Internet, Transport, and Application. OSI is a reference model; real protocols do not always map perfectly one-to-one.

Protocols, addressing and devices

- **TCP** is connection-oriented and provides reliable, ordered byte-stream delivery. **UDP** has lower overhead but does not guarantee delivery or order. TCP uses acknowledgements, retransmission, flow control, and congestion control.
- IPv4 uses 32-bit addresses; IPv6 uses 128-bit addresses. CIDR prefix /24 means 24 bits identify the network. DNS maps names to records; DHCP leases configuration; ARP resolves IPv4 addresses to local link-layer addresses.
- Switches forward LAN frames using MAC addresses; routers forward packets between IP networks; access points connect wireless devices. HTTP/HTTPS serve web traffic; SMTP sends email; IMAP/POP3 retrieve it; SSH provides secure remote access.

Performance and troubleshooting

- **Bandwidth** is capacity; **throughput** is achieved data rate; **latency** is delay; **jitter** is delay variation; **packet loss** is missing traffic.
- Common default ports include HTTP 80, HTTPS 443, DNS 53, SSH 22, and SMTP submission 587. Defaults can be changed by configuration.
- Troubleshoot layer by layer: physical link, IP configuration, gateway, DNS, routing, port reachability, and application logs. Tools include ping, traceroute, ipconfig/ifconfig, nslookup/dig, and netstat/ss.

4. Artificial Intelligence and Machine Learning

AI studies systems that perform tasks associated with intelligent behavior. ML is a subset of AI in which models learn patterns from data.

AI foundations and search

- AI areas include search and planning, knowledge representation, rule-based reasoning, probabilistic reasoning, NLP, computer vision, robotics, and machine learning.
- Breadth-first search explores by depth; depth-first search follows a branch; uniform-cost search expands the lowest-cost path. A* evaluates f(n)=g(n)+h(n), where g is cost so far and h estimates remaining cost. An admissible heuristic never overestimates the true remaining cost.
- Minimax is used for adversarial games; alpha-beta pruning avoids exploring branches that cannot affect the final minimax decision.

Types of machine learning

- **Supervised learning:** labeled examples; used for classification and regression. **Unsupervised learning:** finds structure without target labels, such as clustering. **Semi-supervised learning:** uses limited labels plus unlabeled data. **Reinforcement learning:** learns through actions and rewards.
- Common algorithms include linear/logistic regression, decision trees, random forests, support vector machines, k-nearest neighbors, k-means, and neural networks. CNNs are common in vision; transformers use attention and are widely used in language and multimodal systems.

Workflow and evaluation

- Workflow: define the problem → gather and clean data → split train/validation/test sets → build a baseline → train and tune → evaluate → deploy → monitor.
- Classification metrics: accuracy = correct/total; precision = TP/(TP+FP); recall = TP/(TP+FN); F1 is the harmonic mean of precision and recall. Regression metrics include MAE, MSE/RMSE, and R^2.
- **Overfitting** means learning noise in training data; **underfitting** means failing to capture patterns. Cross-validation, regularization, early stopping, and more representative data can help. Data leakage makes evaluation unrealistically optimistic.

Generative AI and responsibility

- Generative models create text, images, audio, code, or other content. Large language models generate tokens based on context and may produce plausible but incorrect outputs.
- Retrieval-augmented generation (RAG) retrieves relevant material and supplies it as context; it can improve grounding but does not guarantee correctness. Responsible AI considers privacy, security, fairness, robustness, transparency, and human oversight.

5. Web Technology

Web technology combines browser-side structure and behavior with server-side processing and network communication.

Frontend basics

- **HTML** defines structure and meaning; semantic elements include header, nav, main, section, article, and footer. Accessible forms need labels, useful input types, keyboard support, and clear errors.
- **CSS** controls presentation. The box model consists of content, padding, border, and margin. The cascade considers origin, importance, specificity, and source order.
- **JavaScript** adds behavior through variables, functions, objects, arrays, DOM APIs, events, modules, and fetch. Use const by default and let when reassignment is needed.
- Flexbox is useful for one-dimensional alignment; Grid handles two-dimensional layouts; media queries and fluid sizing support responsive design.

HTTP, APIs and backend

- Typical flow: URL entry → DNS lookup → connection → TLS when secure → HTTP request → server response → browser parses and renders resources.
- HTTP methods: GET retrieves, POST submits/creates, PUT replaces, PATCH partially updates, DELETE removes. Status code families: 2xx success, 3xx redirect, 4xx client-side issue, 5xx server-side issue.
- A backend handles business logic, authentication, authorization, database access, and integrations. REST APIs commonly expose resources through HTTP; JSON is a common data-exchange format. Validate all input on the server.
- Cookies can be sent with requests according to their scope and policy. localStorage persists key-value data; sessionStorage is scoped to a tab/session. Avoid placing secrets in browser storage.

Tooling and web security

- Git tracks changes; branches isolate work; commits create snapshots; pull requests enable review. npm manages packages and scripts. Keep secrets out of source control and use environment configuration carefully.
- Testing includes unit, integration, end-to-end, accessibility, performance, and security tests. Browser developer tools inspect DOM, CSS, network requests, storage, and console errors.
- Prevent SQL injection with parameterized queries. Prevent XSS with context-aware output encoding and safe DOM APIs. Protect cookie-authenticated state changes against CSRF. Use HTTPS, secure sessions, least privilege, rate limits, and dependency updates.

6. Network Security and Cryptography

Security protects confidentiality, integrity, availability, authenticity, and accountability. Cryptography is one part of a secure system; key management and operations are equally important.

Goals and threats

- **Confidentiality:** only authorized parties can read data. **Integrity:** unauthorized changes are detectable. **Availability:** services remain accessible. Authentication verifies identity; authorization controls permitted actions.
- Threats include phishing, malware, ransomware, credential theft, eavesdropping, spoofing, replay, man-in-the-middle attacks, denial of service, privilege escalation, and misconfiguration.
- Defense in depth includes patching, MFA, least privilege, segmentation, backups, monitoring, endpoint protection, secure defaults, and incident response.

Cryptographic building blocks

- **Symmetric encryption:** the same secret key encrypts and decrypts; AES is a common standard. **Asymmetric cryptography:** public/private key pairs support key agreement and signatures; RSA and elliptic-curve systems are examples.
- **Hash:** fixed-size digest used as a fingerprint; SHA-256 is a common hash. **MAC/HMAC:** checks integrity and authenticity using a shared secret. **Digital signature:** private key signs and public key verifies.
- Hashing is not encryption and is not designed to be reversed. Passwords should be stored using salted, deliberately expensive password-hashing schemes such as Argon2id, scrypt, or PBKDF2—not plain hashes.

TLS and network defenses

- Authenticated encryption modes such as AES-GCM and ChaCha20-Poly1305 can provide confidentiality and integrity. Use vetted libraries; do not invent cryptographic algorithms or reuse nonces where prohibited.
- TLS protects data in transit and commonly authenticates servers using certificates. PKI uses certificate authorities to bind identities to public keys; certificate and hostname validation are essential.
- Firewalls filter traffic; IDS detects suspicious activity; IPS may block it; VPNs create protected tunnels. Segmentation limits lateral movement. Keep logs, patch systems, test backups, and prepare a response plan.

7. Data Structures and Algorithms (DSA)

Data structures organize information; algorithms solve problems using those structures. Analyze correctness as well as time and memory costs.

Complexity

- Time complexity describes how operation count grows with input size n; space complexity describes memory growth. Big-O is an asymptotic upper bound, Big-Ω a lower bound, and Big-Θ a tight bound.
- Common growth rates: O(1), O(log n), O(n), O(n log n), O(n^2), O(2■), O(n!). Best, average, and worst cases may differ. Amortized analysis studies the cost over a sequence of operations.

Core data structures

- Array/dynamic array: indexed access O(1), but middle insertion/deletion O(n). Linked list: insert/delete at a known node can be O(1), but indexed access is O(n).
- Stack is LIFO; queue is FIFO. Hash tables provide average O(1) lookup/insert but can degrade with collisions. Balanced search trees offer O(log n) operations. Heaps support peek-min/max in O(1) and insert/remove in O(log n).
- Graphs represent relationships using vertices V and edges E. Adjacency lists are often space-efficient for sparse graphs; adjacency matrices offer O(1) edge checks but use O(V^2) space.

Search, sort, trees and graphs

- Linear search is O(n); binary search is O(log n) but requires sorted data. Bubble, selection, and insertion sort are generally O(n^2). Merge sort and heap sort are O(n log n); quicksort is average O(n log n) but worst O(n^2).
- Tree traversals include preorder, inorder, postorder, and level order. BFS explores by layers and finds shortest paths in unweighted graphs. DFS supports cycle detection, connected components, and backtracking; both can run in O(V+E) with adjacency lists.
- Dijkstra finds shortest paths with nonnegative weights; Bellman-Ford handles negative edges and detects negative cycles. Kruskal and Prim find minimum spanning trees. Topological sorting applies to directed acyclic graphs.

Problem-solving patterns

- Common approaches: two pointers, sliding window, prefix sums, binary search on an answer, recursion, backtracking, greedy algorithms, divide and conquer, dynamic programming, and union-find.
- Dynamic programming works well when subproblems overlap and a larger solution can be built from smaller ones. Define the state, recurrence, base cases, and evaluation order.
- Test edge cases: empty input, one item, duplicates, sorted/reversed data, extreme values, disconnected graphs, cycles, and invalid input.

8. Operating Systems (OS)

An operating system manages hardware and provides services for processes, memory, files, devices, and security.

Processes, threads and IPC

- A program is passive code; a process is a running instance with resources and an address space. A thread is an execution path within a process; threads generally share process memory.
- Common process states are new, ready, running, waiting/blocked, and terminated. Context switching saves and restores execution state. IPC mechanisms include pipes, message queues, shared memory, sockets, and signals.
- Shared data can create race conditions. Mutexes, semaphores, monitors, and atomic operations coordinate access to critical sections.

CPU scheduling

- **FCFS:** first come, first served; simple but may cause convoy effects. **SJF/SRTF:** shortest job/remaining time first; good average waiting time but needs burst estimates. **Priority:** highest priority first; aging can reduce starvation.
- **Round Robin:** each ready process receives a time quantum; useful for interactive systems. A quantum that is too large behaves like FCFS; one that is too small creates excess context-switch overhead. Multilevel feedback queues use multiple queues and changing priorities.

Deadlocks and memory

- A deadlock requires four Coffman conditions: mutual exclusion, hold and wait, no preemption, and circular wait. Prevention breaks a condition; avoidance checks safe states; detection and recovery allow deadlocks to occur and then resolve them.
- Paging divides virtual memory into fixed-size pages mapped to physical frames. Segmentation uses logical variable-sized regions. Virtual memory lets processes use virtual address spaces backed by RAM and storage.
- A page fault occurs when a needed page is not currently resident/mapped. Replacement algorithms include FIFO, LRU, and optimal (a theoretical benchmark). Thrashing occurs when excessive paging prevents useful progress.

Files, I/O and protection

- File systems manage files, directories, metadata, permissions, allocation, and recovery. I/O can use polling, interrupts, or direct memory access (DMA).
- User mode restricts privileged operations; kernel mode runs trusted OS code. System calls provide controlled access to OS services. Permissions, isolation, least privilege, and patching support protection.

9. Object-Oriented Programming (OOP)

OOP organizes software around objects that combine state and behavior. Language details differ, but the core design ideas are widely shared.

Four common pillars

- **Encapsulation:** bundle state and behavior and control access to internal details. Example: a bank account exposes deposit/withdraw methods instead of allowing arbitrary balance changes.
- **Abstraction:** expose essential behavior while hiding implementation. Interfaces and abstract classes can define contracts.
- **Inheritance:** derive a class from another to reuse or specialize behavior. Use it when the subtype relationship is valid, not merely to avoid duplicate code.
- **Polymorphism:** one interface can have multiple implementations. Method overriding and method overloading are distinct mechanisms.

Classes, objects and design

- A class defines a type or blueprint; an object is an instance. Fields/properties store state; methods implement behavior; constructors establish initial state. Static members belong to a class/type rather than one instance.
- Association means objects are related; aggregation is a weaker whole-part relationship; composition usually implies stronger ownership/lifetime coupling. Exact terminology varies by language/textbook.
- SOLID: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Prefer composition when behavior can be assembled from collaborators; reduce coupling and keep responsibilities clear.
- Conceptual example: Circle and Rectangle both implement an area() operation. Code can call area() through a common Shape contract without checking each concrete class.

10. Software Engineering

Software engineering applies systematic methods to requirements, design, implementation, testing, deployment, and maintenance.

SDLC and development approaches

- Typical life cycle: feasibility/planning → requirements → design → implementation → testing → deployment → operation and maintenance. Teams may revisit stages iteratively.
- **Waterfall** emphasizes sequential phases and defined baselines. **Agile** emphasizes iterative delivery and feedback. Scrum organizes work with defined accountabilities, events, and artifacts; Kanban visualizes flow and limits work in progress.

Requirements, design and testing

- Functional requirements describe what a system does. Non-functional requirements describe qualities/constraints such as performance, security, accessibility, reliability, and usability. Use cases, user stories, prototypes, and acceptance criteria help clarify expectations.
- Good architecture uses modularity, high cohesion, low coupling, separation of concerns, and clear interfaces.
- Unit tests check small components; integration tests check interactions; system tests check the complete system; acceptance tests check business/user needs; regression tests check that changes did not break existing behavior.
- Black-box testing derives tests from behavior; white-box testing considers internal structure. Boundary-value analysis and equivalence partitioning help choose efficient tests. Verification asks whether the product follows specifications; validation asks whether it meets real needs.

Project management and maintenance

- Estimate scope, effort, cost, schedule, risk, and dependencies. Risk management records probability, impact, mitigation, and contingency.
- Version control tracks changes; continuous integration (CI) builds and tests changes; continuous delivery/deployment automates release processes with appropriate safeguards.
- Maintenance can be corrective (fix defects), adaptive (new environment), perfective (improve features/performance), or preventive (reduce future risk). Technical debt is a trade-off that should be visible and managed.

11. Compiler Design

A compiler translates source code into another representation, often machine code, while checking language rules and preserving program meaning.

Compiler phases

- **Lexical analysis:** character stream to tokens. **Syntax analysis:** checks grammar and builds a parse tree/AST. **Semantic analysis:** checks declarations, types, scope, and language rules.
- **Intermediate representation (IR):** machine-independent form. **Optimization:** improves speed/size while preserving behavior. **Code generation:** emits assembly or machine code. Linking resolves symbols; loading prepares execution.
- A token is a category such as identifier or keyword; a lexeme is the actual character sequence. Regular expressions and finite automata are common lexer tools.

Grammars, semantic analysis and IR

- Context-free grammars use terminals, nonterminals, productions, and a start symbol. Parsing families include top-down recursive descent/LL and bottom-up LR methods. Ambiguous grammars allow multiple parse trees for a string.
- Symbol tables track names, types, scopes, and attributes. Type checking rejects invalid operations. An abstract syntax tree captures meaningful program structure.
- Three-address-code example: t1 = b * c; t2 = a + t1. Intermediate forms may include control-flow graphs and static single assignment (SSA).

Optimization and code generation

- Common optimizations include constant folding/propagation, dead-code elimination, common-subexpression elimination, loop-invariant code motion, and strength reduction.
- Optimizations must preserve observable behavior, including exceptions, memory behavior, and language rules. Code generation selects instructions and addressing modes; register allocation maps values to limited hardware registers, spilling some values to memory.
- An interpreter executes code or an intermediate representation; a compiler translates before execution. Modern systems may combine interpretation, compilation, and just-in-time (JIT) optimization.

12. Computer Organization and Architecture (COA)

Architecture describes programmer-visible behavior and the instruction set; organization describes how hardware implements it.

Computer structure and number systems

- The CPU contains a control unit, ALU, registers, and often caches. Main memory stores instructions and data; controllers and buses/interconnects connect components.
- The fetch-decode-execute cycle fetches an instruction, decodes it, executes the operation, and updates state. Common registers include program counter (PC), instruction register (IR), stack pointer (SP), and status flags.
- Binary is base 2; hexadecimal is base 16 and compactly represents groups of four binary bits. Unsigned n-bit values range 0 to 2■–1. Two's-complement signed n-bit values range –2■■¹ to 2■■¹–1.
- Floating-point numbers have finite precision. Overflow occurs when a result cannot be represented in the selected format.

Instruction sets and memory hierarchy

- An instruction set architecture (ISA) defines instructions, registers, data types, memory model, and exceptions. Examples include x86-64 and ARM. Addressing modes include immediate, register, indirect, indexed, and base-plus-offset.
- Memory hierarchy: registers → L1/L2/L3 cache → RAM → SSD/HDD. Faster storage is generally smaller and more expensive per bit. Caches exploit temporal locality (reuse soon) and spatial locality (nearby addresses).
- Cache mapping may be direct-mapped, fully associative, or set-associative. Misses may be compulsory, capacity, or conflict. Write policies include write-through and write-back.

Performance, pipelining and I/O

- CPU time ≈ instruction count × CPI × clock-cycle time. Clock frequency alone does not determine performance; instruction count, CPI, memory stalls, and parallelism matter.
- Amdahl's law: if fraction p of a workload is improved by factor s, speedup = 1 / ((1–p) + p/s). The unimproved portion limits total speedup.
- Pipelining overlaps instruction stages to improve throughput. Hazards are structural, data, and control hazards; forwarding, stalls, and branch prediction can mitigate them.
- DMA transfers blocks between devices and memory with less CPU involvement; interrupts notify the CPU of events. Parallelism may use multiple cores, SIMD/vector operations, GPUs, or distributed computers.

Quick Revision Checklist

- ■ DBMS: keys, joins, normalization, ACID, indexes, transactions.
- ■ Data warehousing: OLTP vs OLAP, ETL/ELT, fact/dimension tables, star schema, SCD.
- ■ Networks: OSI/TCP-IP, subnetting, TCP vs UDP, DNS/DHCP, routing, latency and throughput.
- ■ AI/ML: search, supervised vs unsupervised learning, train/test split, metrics, overfitting, leakage.
- ■ Web: HTML/CSS/JS roles, HTTP methods/status codes, APIs, responsive design, web security.
- ■ Security: CIA triad, symmetric/asymmetric crypto, hashing vs encryption, TLS, key management.
- ■ DSA: Big-O, arrays/lists/hash tables/trees/graphs, sorting, BFS/DFS, shortest paths, DP.
- ■ OS: process vs thread, scheduling, synchronization, deadlocks, paging, virtual memory.
- ■ OOP: encapsulation, abstraction, inheritance, polymorphism, composition, SOLID.
- ■ Software engineering: requirements, SDLC, testing levels, quality, CI/CD, maintenance.
- ■ Compiler design: lexical, syntax, semantic analysis, IR, optimization, code generation.
- ■ COA: number systems, CPU cycle, ISA, cache hierarchy, pipelining, CPU performance equation.

Mini glossary

Term	Meaning
API	A defined interface through which software components communicate.
Cache	Small, fast storage that keeps likely-to-be-reused data close to a processor or service.
Concurrency	Multiple tasks making progress during overlapping time periods.
Encryption	Reversible transformation using a key to protect confidentiality.
Index	Auxiliary structure that speeds up selected database queries.
Latency	Time delay between a request/action and its response.
Protocol	Rules that define how communicating systems format and exchange messages.
Transaction	A logical unit of database work governed by transaction rules.
Virtual memory	An abstraction providing virtual address spaces backed by physical memory and storage.

End of guide • Keep learning through practice and projects.